Randomized Identity Checking of Polynomials

From 'An Introduction to Mathematics' Exposition

> A fundamental question
>
> Is it easy (more fundamentally, is it possible) to algorithmically decide whether two given functions are equal?

I believe that trying to answer this question is a good way to revisit our understanding of several concepts in mathematics.

- What is a function, and when are two functions equal? A high school student might only think of a function as a rule for assigning an input number to an output number, explicitly defined by an algebraic expression. The point at which one finally is able to comfortably think of and work with functions in the abstract formulation provided by set theory is a memorable moment in one's mathematical journey, and it has actually been a major historical moment in the history of mathematics.
- What do we mean by "deciding algorithmically"?
- More importantly, what do we mean by "is it possible"? It seems that algorithms are the almighty creatures of our time. An algorithm, as people often say, can recognize a face when unlocking a phone, can drive a car, can beat a human in chess, and can compute an integral. Are there things that algorithms cannot do? Aren't we human beings just a bunch of algorithms?
- What is the meaning of "easy" in this context?

These are some obvious questions that come to my mind when I think about the above question. There are of course a lot of subtilities that may arise as well. My favorite is the following:

- The algorithm that is asked for has to take as input two functions and decide whether they are equal. How are the functions given? Does it have an impact on the answer to the question?

Among the few classes of functions that a high school student might be familiar with, polynomials are the most familiar. They are a respected family of functions. They are quite polite and well-behaved, especially when compared to some rebellious functions one usually encounters in a calculus course. The extra structures that polynomials are endowed with usually make them easier to reason about. So, a good starting point for answering our question might be restricting our attention to polynomials.

> In this exposition, I am restricting my attention to univariate polynomials over the field of real or complex numbers.

If we are given the list of coefficients of two polynomials, and let's say that the polynomials are of degree n, then we can decide whether they are equal by comparing the coefficients of the corresponding terms. This relies on the fact that two polynomials of degree n are equal if and only if the coefficients of their corresponding terms are equal.

What if, however, we are not given the list of coefficients of both polynomials, but instead we are given the list of coefficients for one, and a list of roots (with multiplicity) for the other?

Exercise 0.1. Show that in the case where we are given the list of coefficients of one polynomial and the list of roots of the other, we can decide whether the two polynomials are equal by $O(n^2)$ monomial multiplications.

The algorithm that you have just described is an algorithm that gives you the correct answer in a deterministic manner. However, keep this important advice of Umesh Vazirani in mind: "If

you have never missed a flight, you are probably spending too much time at the airport." Is it possible to allow for some probability of error in our algorithm, and we get a faster algorithm in return?

Here is a possible faster randomized algorithm[1] for the problem:

[1] Randomized algorithms that have a fixed running time but may output an incorrect answer with some probability are called Monte Carlo algorithms.

Input: Two polynomials $P(x)$ and $Q(x)$ of degree d, where $P(x)$ is given by its coefficients and $Q(x)$ is given by its roots.

1. Choose an arbitrary integer k.
2. For $i = 1, 2, \dots, k$:
   - Choose r_i uniformly at random from the set $\{0, 1, \dots, d\}$.
   - If $P(r_i) \neq Q(r_i)$, output "Not Equal" and halt.
3. Output "Equal".

Exercise 0.2. Show that if the polynomials are equal, the algorithm above always outputs "Equal".

Exercise 0.3. Justify that if the algorithm above outputs "Not Equal", then the polynomials are not equal.

The above exercises show that there is no error in the output of the algorithm when it returns "Not Equal".[2] However, the algorithm might return "Equal" even if the polynomials are not equal. Let us compute the probability of this happening.

[2] This property is usually referred to as having one-sided error.

If $P \neq Q$, then there exists an element $r \in \{0, 1, \dots, d\}$ such that $P(r) \neq Q(r)$, otherwise the polynomials would be equal. Assume that the algorithm outputs "Equal". We want to upper bound the probability of this event.

$$\mathbb{P}\left(\text{Algorithm outputs "Equal"}\right) \leq \mathbb{P}\left(r_1 \neq r, r_2 \neq r, \dots, r_k \neq r\right) = \left(1 - \frac{1}{d+1}\right)^k.$$

Thus,

$$\mathbb{P}\left(\text{Algorithm outputs "Not Equal"}\right) = 1 - \left(1 - \frac{1}{d+1}\right)^k.$$

Note that the probability of error decreases exponentially with the number of repetitions k, and the probability of correctly answering the question in the case where the polynomials are not equal tends to 1 as k grows. This exponential decrease in the probability of error is what makes the algorithm efficient.

Exercise 0.4. Show that $k \in O(\log(1/\epsilon))$ repetitions suffice to achieve a probability of error at most ϵ.